

React / Next.js Coding Standard — v2.0

Owner: Jegadhesh · **Approved:** Praveen (CTO)

Applies to: all React projects. Next.js App Router is the default; Appendix C covers non-Next React.

Supersedes: React Coding Standard v1.0

Scope and status

This standard applies to **new and changed code**. Existing code is not retrofitted — a rule arriving here does not create a backlog of rewrites. When you touch a file, the code you write in it follows this document.

Where a rule has sample code, that code is the house pattern. Copy it rather than writing your own version.

Sections 1–13 mirror the Angular standard section for section, so a developer moving between the two teams finds the same rule in the same place. Section 14 is React-specific.

Version and library policy

Projects run the latest stable React and the latest LTS Node for tooling. Upgrades are planned work, scheduled at least once per major cycle, not deferred until something forces them.

Fixed library choices for all new projects:

Concern	Choice
Framework	Next.js, App Router
Server state	TanStack Query for client-interactive data (section 14)
Styling	Tailwind (section 13.2)
Validation	Zod at every API boundary
Forms	react-hook-form with a Zod resolver
Error tracking	Sentry

These are not per-project preferences. A developer moving between React projects should not have to relearn how data is fetched.

Pages Router projects are not migrated. They keep Pages Router and follow every rule here that applies to them. New projects use the App Router.

Enforcement tags

Tag	Meaning
[LINTER]	A tool in the repo catches it — ESLint, react-hooks, testing-library, tsc .
[CI]	A pipeline step or Git-host setting catches it — Gitleaks, dependency scan, coverage gate, branch protection.
[NEITHER]	A human must judge it in review.
[LINTER + NEITHER]	The linter covers part; review covers the rest.
[CI + NEITHER]	The pipeline covers part; review covers the rest.

Check: notes name the specific tool, or explain why a rule is not automatable. A **backstop** is a narrower rule a tool *can* enforce, used where the real rule cannot be automated.

Roughly 45% of this document is [NEITHER] . That is the measured reason human review is mandatory on every PR.

1. Naming

- [LINTER + NEITHER] Component files are `PascalCase.tsx` matching their export (`UserProfile.tsx`); everything else is `camelCase.ts` (`useAuth.ts`, `apiClient.ts`). Folders are kebab-case. **Check:** Linter — `eslint-plugin-check-file`. Human — whether the file name matches its export.
- [LINTER] Custom hooks always start with `use`. *Reason:* the lint rules that catch conditional hook calls depend on that prefix. **Check:** `react-hooks/rules-of-hooks`.
- [LINTER] Props that take a function are `onSomething`; the implementation inside the component is `handleSomething`. *Reason:* `onSave` outside and `handleSave` inside makes the direction of the call obvious. **Check:** `react/jsx-handler-names`.
- [LINTER + NEITHER] Booleans read as assertions: `isLoading`, `hasAccess`, `canSubmit`. No `data`, `info`, `temp`, `flag`, `item2` as a name — if that is the best name available, the variable is doing too much. **Check:** Linter — `@typescript-eslint/naming-convention`, `id-denylist`. Human — names that pass the rule but still say nothing.
- [LINTER] Constants `UPPER_SNAKE_CASE`. Types and interfaces `PascalCase`, no `I` prefix. **Check:** `@typescript-eslint/naming-convention`.

1.1 Wire format across stacks

[NEITHER] JSON payloads are camelCase on the wire, in both directions.

Our Python services use `snake_case` internally and our Node services use `camelCase`. Both are correct in their own language. The contract between them is `camelCase`.

Python converts at the boundary with Pydantic field aliases. React never converts: what arrives from the API is already in the shape the Zod schema declares.

If an endpoint returns `snake_case`, that is a backend defect — raise it rather than mapping around it. A per-feature mapping layer is how one inconsistent endpoint becomes twelve.

2. Project structure

`app/` holds routing only. Feature code lives outside it. *Reason:* routing is Next's concern and changes with the URL structure; a feature is what gets built, reviewed and deleted as a unit.

```
src/
  app/                                # ROUTING ONLY — thin pages that compose features
    layout.tsx                       # Server Component. Never 'use client'.
    global-error.tsx
    (auth)/login/page.tsx
    (app)/
      layout.tsx
      loans/
        page.tsx                   # Server Component: reads params, renders the feature
        loading.tsx
        error.tsx
        [id]/page.tsx
    api/                             # Route Handlers — thin proxy only, see 8.6
  features/
    loans/
      api/                         # queries, mutations, schemas, query keys
        components/
        hooks/
        types.ts
    components/                    # shared presentational UI, no business logic
    hooks/                         # genuinely global hooks
    lib/                           # api client, config, logging, formatters
    styles/                        # tokens.css and the global layer
```

- [NEITHER] A file in `app/` is a route, not a feature. A `page.tsx` reads params, checks access, and renders a component from `features/`. Business logic, data shaping and layout detail do not live in `app/`. *Reason:* when the URL structure changes — and on client work it does — only `app/` moves.
- [NEITHER] Feature-first on every new project. No exceptions. The layout costs nothing on a small project. Existing projects on another layout are not migrated.
- [NEITHER] Used by two or more features, it moves to `components/` or `hooks/`. Used once, it stays inside the feature.
- [LINTER] No cross-feature imports. Features talk through `lib/` or shared components, never to another feature's internals. **Check:** `eslint-`

plugin-boundaries.

- [NEITHER] Route groups (auth) and (app) separate the signed-out and signed-in shells. They do not appear in the URL.

2.1 The 'use client' boundary

This is the most consequential rule in a Next codebase, and the easiest to get wrong.

[NEITHER] Server Component is the default. 'use client' is opt-in, placed as far down the component tree as possible.

Reason: 'use client' is not a per-file switch — it marks a **boundary**. Every component imported below it also becomes a client component. Putting it at the top of a `page.tsx` ships that entire page, and everything it renders, to the browser as one bundle. Our own standard names bundle size as the most visible performance number we have.

```
// ✖ The whole page and every child become client code.
'use client';
export default function LoansPage() {
  const [filter, setFilter] = useState('');
  return (
    <div>
      <PageHeader title="Loans" />           {/* now client code for no reason */}
      <LoanFilters value={filter} onChange={setFilter} />
      <LoanList filter={filter} />
    </div>
  );
}
```

```
// ✔ The page stays on the server. Only the interactive part crosses the boundary.
export default function LoansPage() {
  return (
    <div>
      <PageHeader title="Loans" />           {/* server-rendered */}
      <LoanBrowser />                       {/* 'use client' lives inside this file */}
    </div>
  );
}
```

Rules:

- [NEITHER] Never 'use client' in `app/layout.tsx` or a route-group layout. A client layout defeats streaming for every page beneath it.
- [NEITHER] Never 'use client' in a `page.tsx` by default. Extract the interactive region into a component that carries the directive itself.
- [NEITHER] A component needs 'use client' only when it uses state, effects, event handlers, browser APIs, or a context provider. Rendering data does not require it.
- [NEITHER] Providers go in one client file, imported by the root layout, so the layout itself stays a Server Component:

```
// src/app/providers.tsx
'use client';
export function Providers({ children, config }: { children: React.ReactNode; config: PublicConfig }) {
  const [queryClient] = useState(() => makeQueryClient());
  return (
    <ConfigProvider value={config}>
      <QueryClientProvider client={queryClient}>{children}</QueryClientProvider>
    </ConfigProvider>
  );
}
```

- [NEITHER] A Server Component may render a client component. A client component cannot import a Server Component — pass it through `children` instead.
- [NEITHER] Browser APIs — `window`, `document`, `localStorage` — do not exist during server rendering. They are read inside `useEffect` or behind a mounted check, never in a render body. Reason: touching them at module or render level crashes the server render, and the error surfaces as a hydration failure that reads as something else entirely.

Wallet and signing screens are necessarily client-heavy. That is a correct exception, not a compromise. Push the boundary as low as the wallet interaction allows and leave the rest of the page on the server.

Check: review item 19. There is no lint rule that judges whether a 'use client' is placed at the right level.

3. Error handling & validation

- [LINTER + NEITHER] Server data goes through one API layer that throws typed errors. Components never call `fetch` directly. **Check:** Linter — `no-restricted-globals` / `no-restricted-imports` banning `fetch` and `axios` outside `lib/`. Human — that the layer throws typed errors.
- [NEITHER] API responses are validated at the boundary with Zod before entering state. *Reason:* an unexpected `null` should fail at the edge with a clear message, not three components deep.
- [NEITHER] Every route segment has an `error.tsx`, and the root has a `global-error.tsx`. *Reason:* without one, a single render error blanks the whole app instead of one screen.
- [NEITHER] Forms use react-hook-form with a Zod resolver. Validation rules live in the schema, not inside `onSubmit`.
- [NEITHER] Users never see a raw error object or backend message. Map to a human message; keep the technical detail in the log.
- [NEITHER] Every async view has four explicit states: **loading, empty, error, success**. A screen that only handles the happy path is not done.

3.1 The error contract

```
// src/lib/api/api-error.ts

/** The body every one of our APIs returns on failure. */
export interface ApiErrorBody {
  code: string; // 'LOAN_ALREADY_SETTLED'
  message: string; // developer-facing; NEVER shown to a user
  details?: Record<string, string[]>; // field name -> validation messages
}

export class ApiError extends Error {
  constructor(
    readonly code: string,
    readonly status: number,
    readonly correlationId: string | null,
    readonly details?: Record<string, string[]>,
    developerMessage?: string,
  ) {
    super(developerMessage ?? code);
    this.name = 'ApiError';
  }
}
```

Branch on `code`, never on `message`. Message text changes; codes are a contract.

```
// src/lib/api/error-messages.ts
const MESSAGES: Record<string, string> = {
  NETWORK_UNAVAILABLE: 'We could not reach the server. Check your connection and try again.',
  SESSION_EXPIRED: 'Your session has ended. Please sign in again.',
  LOAN_ALREADY_SETTLED: 'This loan has already been settled.',
};

const FALLBACK = 'Something went wrong. Please try again.';

export function userMessage(error: unknown): string {
  return error instanceof ApiError ? MESSAGES[error.code] ?? FALLBACK : FALLBACK;
}
```

Unmapped codes fall back deliberately. **Never show `error.message` to a user** — it is written for developers and may carry internal detail.

3.2 The API client

One module. Everything goes through it. It attaches the correlation ID and the token, validates the response, and coordinates token refresh.

```
// src/lib/api/client.ts
import { z, type ZodType } from 'zod';
import * as Sentry from '@sentry/react';

/** The one in-flight refresh, or null. See 8.4. */
let refreshPromise: Promise<string> | null = null;

async function refreshAccessToken(): Promise<string> {
  if (refreshPromise) return refreshPromise; // join the running refresh

  refreshPromise = (async () => {
```

```

    try {
      const res = await fetch(`${config().apiBaseUrl}/auth/refresh`, {
        method: 'POST',
        credentials: 'include', // sends the httpOnly refresh cookie
      });
      if (!res.ok) throw new ApiError('SESSION_EXPIRED', res.status, null);
      const { accessToken } = (await res.json()) as { accessToken: string };
      tokenStore.set(accessToken);
      return accessToken;
    } catch (e) {
      tokenStore.clear();
      Sentry.setUser(null);
      window.location.assign('/login');
      throw e;
    } finally {
      refreshPromise = null;
    }
  })();

  return refreshPromise;
}

interface RequestOptions<T> extends Omit<RequestInit, 'body'> {
  body?: unknown;
  schema?: ZodType<T>;
}

export async function apiFetch<T = unknown>({
  path: string,
  options: RequestOptions<T> = {},
  allowRetry = true,
}): Promise<T> {
  const { schema, body, headers, ...init } = options;
  const correlationId = crypto.randomUUID();
  const token = tokenStore.get();

  let res: Response;
  try {
    res = await fetch(`${config().apiBaseUrl}${path}`, {
      ...init,
      credentials: 'include',
      headers: {
        'Content-Type': 'application/json',
        'X-Correlation-Id': correlationId,
        ...(token ? { Authorization: `Bearer ${token}` } : {}),
        ...headers,
      },
      body: body === undefined ? undefined : JSON.stringify(body),
    });
  } catch {
    // Network failure, CORS, or the request was blocked.
    logger.error('Network request failed', { path, correlationId });
    throw new ApiError('NETWORK_UNAVAILABLE', 0, correlationId);
  }

  // 401 -> one shared refresh, then retry exactly once.
  if (res.status === 401 && allowRetry) {
    await refreshAccessToken();
    return apiFetch<T>(path, options, false);
  }

  if (!res.ok) {
    const errBody = (await res.json().catch(() => null)) as ApiErrorBody | null;
    const apiError = new ApiError(
      errBody?.code ?? 'UNKNOWN_ERROR',
      res.status,
      res.headers.get('X-Correlation-Id') ?? correlationId,
      errBody?.details,
      errBody?.message,
    );
    // 4xx is usually the user's problem. 5xx is ours.
    if (res.status >= 500) {
      logger.error('Server error', { path, status: res.status, code: apiError.code, correlationId });
    }
    throw apiError;
  }

  if (res.status === 204) return undefined as T;

  const json: unknown = await res.json();
  if (!schema) return json as T;

  const parsed = schema.safeParse(json);
  if (!parsed.success) {

```

```

// The contract broke. Fail here, with the shape, not three components deep.
logger.error('Response failed schema validation', {
  path, correlationId, issues: parsed.error.issues,
});
throw new ApiError('INVALID_RESPONSE', res.status, correlationId);
}
return parsed.data;
}

```

The retry is bounded by the `allowRetry` flag. A second 401 throws rather than recursing.

3.3 Schemas at the boundary

```

// src/features/loans/api/schemas.ts
import { z } from 'zod';

export const loanSchema = z.object({
  id: z.string().uuid(),
  principalMinor: z.string(), // money as a string – see section 12
  status: z.enum(['ACTIVE', 'SETTLED', 'DEFAULTED']),
  dueDate: z.string().datetime(),
});

export const loanListSchema = z.array(loanSchema);

export type Loan = z.infer<typeof loanSchema>; // the type comes FROM the schema

```

`z.infer` means the schema is the single definition. A hand-written interface alongside a schema will drift.

3.4 Error and loading files

Next provides these per route segment. Use them rather than hand-rolling boundaries.

```

// src/app/(app)/loans/error.tsx
'use client'; // error files are always client components
import { useEffect } from 'react';
import * as Sentry from '@sentry/nextjs';

export default function LoansError({
  error,
  reset,
}: { error: Error & { digest?: string }; reset: () => void }) {
  useEffect(() => {
    Sentry.captureException(error, { tags: { digest: error.digest } });
  }, [error]);

  return (
    <div role="alert" className="p-8 text-center">
      <h2 className="text-xl font-semibold">This page could not be displayed.</h2>
      {error.digest && <p className="mt-2 text-sm text-text-muted">Reference: <code>{error.digest}</code></p>}
      <button onClick={reset} className="mt-4 rounded-md bg-primary px-4 py-2 text-white">
        Try again
      </button>
    </div>
  );
}

```

```

// src/app/(app)/loans/loading.tsx – shown while the segment streams
export default function Loading() {
  return <LoanListSkeleton />;
}

```

- [NEITHER] `error.tsx` catches errors **below** it, not in the layout at its own level. The root `global-error.tsx` is the only thing that catches a root layout failure, and it must render its own `<html>` and `<body>`.
- [NEITHER] **Show error.digest**. In production Next replaces the server error message with a digest, and that digest is what correlates the screen to the server log. Without it you have a user saying “it broke” and nothing else.
- [NEITHER] A `loading.tsx` with a skeleton beats a spinner. It also means the page streams rather than blocking.

3.5 The canonical async view

Every screen that loads data handles four states, no exceptions. Where a `loading.tsx` and an `error.tsx` exist, the first two are handled by the framework and the component covers empty and success. Where data is fetched client-side, the component covers all four:

```
// src/features/loans/LoanListPage.tsx
export function LoanListPage() {
  const { data, isPending, isError, error } = useLoans();

  if (isPending) return <Spinner />;

  if (isError) {
    return (
      <ErrorState message={userMessage(error)}>
        {error instanceof ApiError && error.correlationId && (
          <small>Reference: <code>{error.correlationId}</code></small>
        )}
      </ErrorState>
    );
  }

  if (data.length === 0) return <EmptyState message="You have no loans yet." />;

  return (
    <ul className="flex flex-col gap-4">
      {data.map(loan => <LoanCard key={loan.id} loan={loan} />)}
    </ul>
  );
}
```

Show the correlation ID on the error state. It costs nothing and turns a support ticket into a log lookup.

4. Logging & observability

- [LINTER + NEITHER] No `console.log` in committed code. Use the logger module. Reason: `console.log` cannot be switched off, filtered or shipped anywhere. Check: `no-console`, with `lib/logging/**` exempted.
- [NEITHER] **Never logged, in any environment:** tokens, passwords, OTPs, API keys, wallet private keys, seed phrases, `Authorization` headers, card or bank data, KYC documents, full API responses containing personal data. Check: backstop — the redaction list enforces this in code (4.2).
- [NEITHER] Production minimum: render errors caught by boundaries and failed API calls reach Sentry with route, user id (never name or email) and correlation id.

4.1 Log levels

The same four meanings across every stack in the company.

Level	Meaning
error	A human needs to look at this.
warn	Degraded but handled.
info	A state change worth auditing.
debug	Off in production.

One `info` line per HTTP call means nobody reads `info` at all.

4.2 Redaction is enforced in code

A denylist in a document is a hope. A denylist in a function is a control. This file is identical to the Angular one — keep them in sync.

```
// src/lib/logging/redact.ts
const DENY = [
  /^authorization$/i, /password$/i, /passwd$/i, /secret$/i, /token$/i,
  /^otp$/i, /^pin$/i, /apikey$/i, /api_key$/i, /privatekey$/i, /private_key$/i,
  /mnemonic$/i, /seed$/i, /^cvv$/i, /cardnumber$/i, /card_number$/i,
  /^aadhaar$/i, /^pan$/i, /^ssn$/i, /^email$/i, /^phone$/i,
];
```

```

const REDACTED = '[REDACTED]';
const MAX_DEPTH = 6;

export function redact(value: unknown, depth = 0): unknown {
  if (depth > MAX_DEPTH) return '[TRUNCATED]';
  if (value === null || typeof value !== 'object') return value;
  if (Array.isArray(value)) return value.map(v => redact(v, depth + 1));

  const out: Record<string, unknown> = {};
  for (const [key, val] of Object.entries(value as Record<string, unknown>)) {
    out[key] = DENY.some(rx => rx.test(key)) ? REDACTED : redact(val, depth + 1);
  }
  return out;
}

```

If something sensitive does reach a log, treat it as a leaked credential and rotate it. The log has already been shipped to Sentry and retained. Removing the log line changes nothing.

```

// src/lib/logging/logger.ts
import * as Sentry from '@sentry/react';

const isProd = () => config().environment === 'production';

export const logger = {
  debug(msg: string, ctx?: unknown) {
    if (isProd()) return;
    console.log(`[debug] ${msg}`, ctx === undefined ? '' : redact(ctx));
  },
  info(msg: string, ctx?: unknown) {
    console.info(`[info] ${msg}`, ctx === undefined ? '' : redact(ctx));
    Sentry.addBreadcrumb({ level: 'info', message: msg, data: redact(ctx) as never });
  },
  warn(msg: string, ctx?: unknown) {
    console.warn(`[warn] ${msg}`, ctx === undefined ? '' : redact(ctx));
    Sentry.addBreadcrumb({ level: 'warning', message: msg, data: redact(ctx) as never });
  },
  /** error = a human needs to look at this. Not "an exception happened". */
  error(msg: string, ctx?: unknown) {
    console.error(`[error] ${msg}`, ctx === undefined ? '' : redact(ctx));
    Sentry.captureMessage(msg, { level: 'error', extra: redact(ctx) as never });
  },
};

```

4.3 Sentry

Sentry initialises **after** config loads, because the DSN comes from `config.json`.

```

// src/lib/observability/sentry.ts
import * as Sentry from '@sentry/react';

export function initSentry(cfg: AppConfig): void {
  if (!cfg.sentryDsn) return; // local development

  Sentry.init({
    dsn: cfg.sentryDsn,
    environment: cfg.environment,
    release: cfg.release,
    tracesSampleRate: cfg.environment === 'production' ? 0.1 : 1.0,
    sendDefaultPii: false,

    // Second line of defence behind redact(). Anything that slips through
    // the logger is scrubbed here before it leaves the browser.
    beforeSend(event) {
      if (event.request?.headers) {
        event.request.headers = redact(event.request.headers) as Record<string, string>;
      }
      if (event.extra) event.extra = redact(event.extra) as Record<string, unknown>;
      if (event.contexts) event.contexts = redact(event.contexts) as never;
      // Query strings carry tokens more often than anyone expects.
      if (event.request?.url) event.request.url = event.request.url.split('?')[0];
      return event;
    },

    beforeSendBreadcrumb(crumb) {
      if (crumb.category === 'console') return null; // already logged
      if (crumb.data) crumb.data = redact(crumb.data) as Record<string, unknown>;
    }
  });
}

```

```

    return crumb;
  },
});
}

/** User id only. Never name, email or phone. */
export const identifyUser = (userId: string) => Sentry.setUser({ id: userId });
export const clearUser = () => Sentry.setUser(null);

```

`sendDefaultPii: false` is not optional. Call `clearUser()` on logout.

5. Testing expectations

The developer proves the code works before it reaches QA. QA validates quality and risk; QA is not the first person to discover whether a feature runs.

- [NEITHER] **Must** have unit tests: custom hooks, reducers, Zod schemas, formatters, and any pure function with a branch.
- [LINTER + NEITHER] Component tests assert behaviour a user can see — text, roles, what happens on click. Not internal state, not implementation details. Snapshot-only tests are not a test. **Check:** Linter — `eslint-plugin-testing-library`, `jest/no-large-snapshots`. Human — whether the test asserts real behaviour.
- [CI] New and changed lines meet the project's coverage threshold; overall coverage must not drop in a PR. *Reason:* a percentage target on legacy code is meaningless; a ratchet on new code is not. **Check:** Vitest/Jest thresholds plus a diff-coverage step.
- [NEITHER] Every fixed bug ships with a test that fails without the fix. **This is our only real regression suite.**
- [LINTER] No `.only` or `fdescribe`; skipped tests need a linked ticket in a comment. **Check:** `vitest/no-focused-tests`, `vitest/no-disabled-tests`, `eslint-comments/require-description`.
- [NEITHER] Tests never run against live third-party APIs, mainnet, or production data. Mock at the network boundary with MSW so the API client is exercised, not stubbed away.
- [CI] Existing tests pass before merge. A developer does not disable, skip or delete a failing test to get a PR through.

6. PR & merge rules

- [CI] **One approval minimum, from someone other than the author, on every PR in every project.** Two approvals when the change touches routing, auth, the API layer or shared providers. **Check:** branch protection plus CODEOWNERS.
- [CI] Blocks merge: failing CI, lint or type errors, self-approval, protected-branch bypass, unresolved blocking comments, dropped coverage.
- [NEITHER] One PR addresses one ticket. Unrelated refactors, formatting sweeps and dependency bumps go in their own PR. *Reason:* a reviewer who cannot tell the fix from the noise approves the noise.
- [NEITHER] Target **400 changed lines**, excluding lock files, generated code and pure renames. Larger PRs carry a note saying why and how to review them.
- [LINTER] Branches: `feature/<ticket>-<slug>`, `fix/<ticket>-<slug>`. Commits describe the change in plain English.
- [CI + NEITHER] Required PR description fields: what changed · why · how it was tested · security/architecture impact · breaking changes · related ticket · deployment requirements. *"Tested locally" is not an answer.*
- [CI] Gitleaks and a dependency scan run on every PR. A detected secret blocks the merge **and triggers rotation of that credential** — once committed it is in the history and must be treated as leaked.

On the review requirement. About 45% of the rules here are [NEITHER] — no tool enforces them. On a project where review is switched off, nearly half this standard does not exist, including most of section 8. That is why the rule is per-company and not per-project. Exceptions are granted by the CTO for a named person, not chosen per project.

7. Review checklist

1. [NEITHER] Does the change match the ticket, and nothing more?
2. [NEITHER] Any `useEffect` that is really derived state, an event handler, or a data fetch that belongs in a query?
3. [LINTER + NEITHER] Correct and complete `useEffect` dependencies, with cleanup where needed? **Check:** `react-hooks/exhaustive-deps`

as an error.

4. [LINTER] `key` on every list item, and is it a stable id rather than the array index? **Check:** `react/jsx-key`, `react/no-array-index-key`.
5. [NEITHER] State that could be computed from existing props or state instead of stored?
6. [LINTER] Any `any`, `as`, or `!` that could be a real type? **Check:** `@typescript-eslint/no-explicit-any`, `no-non-null-assertion`, `consistent-type-assertions`.
7. [NEITHER] Loading, empty, error and success states all handled?
8. [NEITHER] Business logic sitting in a component that belongs in a hook or `lib/`?
9. [LINTER + NEITHER] Hardcoded strings, URLs, IDs or magic numbers? **Check:** `@typescript-eslint/no-magic-numbers`; URLs and UI strings judged by a human.
10. [NEITHER] New dependency, and is it justified in the description?
11. [NEITHER] Anything sensitive reaching a log, a URL, or Sentry?
12. [LINTER] Commented-out code, leftover TODOs, unused imports? **Check:** `sonarjs/no-commented-code`, `no-warning-comments`, `@typescript-eslint/no-unused-vars`.
13. [NEITHER] **Money handled per section 12** — no `number` arithmetic on an amount?
14. [NEITHER] Can it be used with a keyboard, and does every control have an accessible name?
15. [LINTER] Any raw colour, spacing or font-size value instead of a token? **Check:** `stylelint-declaration-strict-value`.
16. [NEITHER] Is a presentational component fetching data, or setting its own outer margin?
17. [NEITHER] Hand-written CSS outside the four permitted places in 13.2?
18. [NEITHER] **Query keys correct, and does the mutation invalidate what it changed?** (section 14)
19. [NEITHER] **Is 'use client' as low in the tree as it can be?** Not on a layout, not on a page by default. (2.1)
20. [NEITHER] Any `window`, `document` or `localStorage` read in a render body rather than an effect?
21. [NEITHER] Does a Route Handler or Server Action contain business logic that belongs in the backend? (8.6)
22. [NEITHER] Authenticated server-side fetch using `cache: 'no-store'`?

8. Security & secrets

8.1 Nothing secret ships to the browser

[CI + NEITHER] No API key, client secret, signing key or credential appears anywhere in the repo or the bundle.

Any variable prefixed `NEXT_PUBLIC_` is public. Treat it as printed on the homepage. Gitleaks will not flag a value someone put there on purpose, which is why this is also a review item.

Next makes a second leak possible that an SPA cannot: **importing a server module into a client component inlines its contents into the browser bundle.** A file that reads `process.env.DB_PASSWORD` and gets imported — directly or transitively — by anything below a `'use client'` boundary ships that value to the browser.

Guard every server-only module:

```
// src/lib/server/secrets.ts
import 'server-only';           // importing this from client code is now a BUILD error

export const stripeSecret = process.env.STRIPE_SECRET_KEY!;
```

[LINTER] Every file under `lib/server/` imports `server-only` on its first line. It converts a silent leak into a failed build.

Prove it once:

```
npm run build
grep -rInE "sk_live|sk_test|AKIA|AIza|xox[baprs]-|----BEGIN" dist/ | head
```

If that prints anything, the key is already public and must be **rotated**, not deleted.

The pattern: when a feature needs a third-party service that requires a key, the browser calls our API and our API calls the third party.

✗ Browser —key→ ThirdParty
✓ Browser —→ Our API —key from AWS Secrets Manager→ ThirdParty

If a vendor's documentation shows a browser-side key, that key is either publishable by design (a Stripe *publishable* key, a Mapbox public token) or the documentation is showing a prototype. **If you are unsure which you are holding, you are holding a secret.**

8.2 Configuration

[NEITHER] One build artifact runs in every environment.

NEXT_PUBLIC_* is baked in at build time. If an image built for dev is promoted to staging and production — which is what we do — those values are wrong in two of the three. So **NEXT_PUBLIC_*** is used only for values that are genuinely identical everywhere.

Everything environment-specific is read **on the server at request time** and passed down.

```
// src/lib/server/config.ts
import 'server-only';

/** Server-only. Never imported below a 'use client' boundary. */
export const serverConfig = {
  apiBaseUrl: process.env.API_BASE_URL!,
  apiKey: process.env.THIRD_PARTY_API_KEY!,
  environment: process.env.APP_ENV as Environment,
};

/** The subset that is safe to send to the browser. */
export function publicConfig(): PublicConfig {
  return {
    environment: serverConfig.environment,
    release: process.env.APP_RELEASE ?? 'dev',
    sentryDsn: process.env.NEXT_PUBLIC_SENTRY_DSN, // a DSN is public by design
    chainId: Number(process.env.CHAIN_ID),
    featureFlags: JSON.parse(process.env.FEATURE_FLAGS ?? '{}'),
  };
}
```

```
// src/app/layout.tsx – Server Component. Reads config, hands the public part down.
import { publicConfig } from '@lib/server/config';

export default function RootLayout({ children }: { children: React.ReactNode }) {
  return (
    <html lang="en">
      <body>
        <Providers config={publicConfig()}>{children}</Providers>
      </body>
    </html>
  );
}
```

- [NEITHER] **publicConfig()** is the allowlist. A value reaches the browser because someone added it there deliberately, not because it happened to be in `process.env`.
- [NEITHER] Client components read config from the context provider, never from `process.env`.

AWS: environment variables come from the task definition or parameter store at container start. The image is identical across environments.

8.3 Session tokens are managed by the backend

[NEITHER] **The session is an httpOnly, Secure, SameSite cookie set by the backend.** JavaScript never reads it.

Where a backend cannot set a cookie, the access token is held **in a module-scoped variable in memory** and nowhere else. Never `localStorage`, never `sessionStorage`, never a JS-readable cookie.

Reason: `localStorage` is readable by any script on the page, including one that arrives through a compromised npm package.

The cookie is also what makes server rendering possible. A Server Component can read it with `cookies()` and fetch on the user's behalf; it cannot see a token held in browser memory. A codebase that keeps tokens in memory is forced to client-render every authenticated screen — that constraint is a consequence of the storage choice, not an architecture decision.

```
// Server-side fetch on behalf of the signed-in user.
import { cookies } from 'next/headers';

export async function serverFetch<T>(path: string, schema: ZodType<T>): Promise<T> {
  const res = await fetch(`${serverConfig.apiUrl}${path}`, {
    headers: { Cookie: (await cookies()).toString() },
    cache: 'no-store', // authenticated data is never cached at the edge
  });
  if (!res.ok) throw new ApiError('REQUEST_FAILED', res.status, null);
  return schema.parse(await res.json());
}
```

[NEITHER] **Authenticated server-side fetches set `cache: 'no-store'`**. A cached response for one user served to another is a data breach, not a performance bug.

```
// src/lib/auth/token-store.ts
let accessToken: string | null = null;

export const tokenStore = {
  get: () => accessToken,
  set: (t: string) => { accessToken = t; },
  clear: () => { accessToken = null; },
};
```

Check: backstop — `no-restricted-properties` on `localStorage` / `sessionStorage` outside one storage module.

8.4 One refresh at a time

[NEITHER] When several requests fail with 401 simultaneously, **exactly one** refresh call is made. Every waiting request joins it and retries once.

Reason: the naive client fires one refresh per failed request. Refresh tokens rotate on use, so the first succeeds and the rest present a consumed token — the backend treats that as replay, revokes the session, and the user is logged out at random. It is intermittent, never reproduces locally, and is usually blamed on the backend.

The implementation is the shared `refreshPromise` in 3.2. **There is one promise, stored at module scope, cleared in `finally`**. Do not reimplement this per feature.

8.5 Hiding a control is UX, not security

[NEITHER] Every protected action is authorised server-side.

Reason: the concrete failure is **IDOR** — a valid token for user A, with user B's id in the URL, returning user B's data. The backend prevents this by scoping the query to the caller. The frontend cannot prevent it and must not be relied on to.

Check: review — for every permission-dependent UI added, name the enforcing API endpoint in the PR description.

8.6 Route Handlers are a proxy, not a second backend

[NEITHER] **A Route Handler in `app/api/` does one of three things and nothing else:** hides a third-party key, sets or clears an `httpOnly` cookie, or receives a webhook.

Reason: Next gives us a server, and the shortest path to shipping a feature is often to write the logic there. That logic then lives outside our Node and Python services, with no review from the backend lead, no tests in their suite, and a second implementation of a rule that already exists. Within a year the two disagree, and nobody knows which is authoritative.

Business logic, validation of domain rules, database access and anything touching money belong in the backend service.

```
// ✓ src/app/api/maps/geocode/route.ts — hides a key, nothing more.
import 'server-only';
import { serverConfig } from '@lib/server/config';

export async function GET(request: Request) {
  const q = new URL(request.url).searchParams.get('q');
  if (!q) return Response.json({ code: 'MISSING_QUERY' }, { status: 400 });

  const res = await fetch(
```

```
`https://maps.example.com/geocode?q=${encodeURIComponent(q)}&key=${serverConfig.mapsKey}``,
);
return Response.json(await res.json(), { status: res.status });
}
```

```
// ✖ This is a backend endpoint that escaped review.
export async function POST(request: Request) {
  const { loanId, amount } = await request.json();
  const interest = amount * 0.025; // business rule, in the frontend repo
  await db.loan.update({ ... }); // direct database access
}
```

[NEITHER] **Server Actions follow the same rule.** They are convenient and they are still frontend code. A Server Action may call our API; it does not replace it.

8.7 User input is never rendered as HTML

[LINTER + NEITHER] No `dangerouslySetInnerHTML` with anything a user can influence. Where it is genuinely needed, the content is sanitised with `DOMPurify` and the call carries an approval comment.

```
// APPROVED BYPASS
// Source: /api/v1/cms/articles – server-side sanitised with DOMPurify before storage.
// Re-sanitised here as defence in depth. Reviewed: <lead name>, <date>. Standard 8.7.
<div dangerouslySetInnerHTML={{ __html: DOMPurify.sanitize(article.html) }} />
```

Wrap it in one `<SafeHtml>` component so there is a single call site to review, rather than one per feature.

Check: Linter — `react/no-danger` flags every use, `eslint-comments/require-description` forces the comment. Human — whether the input is actually sanitised.

8.8 File uploads

[NEITHER] Client-side size and type checks are UX. The server re-validates by inspecting content, stores privately, and serves through expiring signed URLs.

```
export async function upload(file: File): Promise<string> {
  if (file.size > 10 * 1024 * 1024) throw new Error('Maximum size is 10 MB');

  // 1. Our API authorises and returns a short-lived presigned PUT URL.
  const { uploadUrl, objectKey } = await apiFetch<PresignResponse>('/uploads/presign', {
    method: 'POST',
    body: { filename: file.name, contentType: file.type, size: file.size },
    schema: presignSchema,
  });

  // 2. Browser PUTs straight to S3. No AWS credentials in the browser.
  await fetch(uploadUrl, { method: 'PUT', body: file, headers: { 'Content-Type': file.type } });

  // 3. Our API verifies the stored object before treating it as valid.
  return objectKey;
}
```

The bucket is private. Reads go through a presigned GET with a short expiry, never a public object URL.

8.9 Web3: the chain is the source of truth

[NEITHER] Private keys never touch application code — signing happens in the user's wallet. The backend never trusts a transaction hash, amount or status reported by the browser.

Reason: a client can submit someone else's hash, an unconfirmed one, or a fabricated success. The backend re-fetches from chain and verifies sender, recipient, amount and confirmation depth before any state changes.

```
// The browser's job ends at "I submitted this hash."
const hash = await walletClient.writeContract({
  address: config().contracts!['exchange'],
  abi: exchangeAbi,
  functionName: 'purchase',
  args: [listingId, amountInBaseUnits], // bigint, base units
});
```

```
});

await apiFetch(`/orders/${orderId}/submitted`, { method: 'POST', body: { hash } });
// The UI waits for OUR backend to confirm, not for the wallet.
```

Contract addresses and chain IDs come from config, never inline. A hardcoded mainnet address in a staging build is how test transactions reach production contracts.

9. Dependencies & configuration

- [NEITHER] A new dependency needs a line in the PR description: what it does, why we cannot do it in ~50 lines ourselves, last release date, bundle cost. *Reason:* every dependency is permanent attack surface and future upgrade work.
- [CI] Lock file always committed; exact versions pinned for direct dependencies. **Check:** `npm ci` fails on a missing or stale lock file; `save-exact=true` in `.npmrc`.
- [CI] `npm audit` / dependency scan passes per CI policy; High findings block, Medium requires triage.
- [NEITHER] Unmaintained packages — no release in roughly two years — are flagged for replacement rather than adopted.
- [NEITHER] Use the stable release, not the newest. The newest version carries the bugs nobody has hit yet.
- [LINTER] Config is read from one config module, never `import.meta.env` scattered across components. **Check:** `no-restricted-syntax` on `import.meta.env` outside `lib/config`. Unused dependencies caught by `knip` or `depcheck` in CI.

10. Maintainability & complexity

- [LINTER] `strict: true` in `tsconfig`. Prettier and ESLint — with `react-hooks` rules as **errors**, not warnings — are the formatting and correctness authority. **Check:** `tsc`, `prettier --check`, ESLint; all three in CI.
- [LINTER] Max ~75 lines per function, max 3 levels of JSX-independent nesting, max ~250 lines per component file. **Check:** `max-lines-per-function`, `max-depth`, `max-lines`.
- [NEITHER] Components render; hooks and `lib/` decide. Data fetching, transformation and business rules move out of the component body. **Check:** backstop — the `fetch` / `axios` restriction in section 3.
- [NEITHER] **Derive, don't store.** If a value can be computed from props or existing state during render, compute it. *Reason:* every extra piece of state is another thing that can fall out of sync.
- [NEITHER] `memo`, `useMemo` and `useCallback` are added when a measurement shows they help, not by default. *Reason:* most of ours are protecting renders that were already cheap.
- [NEITHER] The third occurrence of duplicated logic gets extracted. The second does not — premature abstraction costs more than duplication.
- [NEITHER] Comments explain **why**, never what. Exported hooks and non-obvious logic carry a short rationale. Every README covers setup, environment variables, and how to run locally and run tests.

11. What we do NOT do

- [LINTER] Array index as a `key` in a list that can reorder, filter or delete. **Check:** `react/no-array-index-key`.
- [NEITHER] `useEffect` to fetch data. That is a query — see section 14.
- [NEITHER] `useEffect` to compute derived state, or to react to a click that already has a handler.
- [NEITHER] `useEffect` with a subscription, timer or listener and no cleanup.
- [NEITHER] Mutating state directly — `state.items.push(x)` then `setState(state.items)`. **Check:** `Readonly` types turn many cases into compile errors.
- [LINTER] `any`. If the shape is unknown, write the type or use `unknown` and narrow. **Check:** `@typescript-eslint/no-explicit-any`.
- [NEITHER] Prop drilling more than about three levels — lift to context or restructure.
- [NEITHER] Context holding a large, frequently-changing object that re-renders every consumer.
- [LINTER] `{...props}` spread onto a DOM element without knowing what is in it. **Check:** `react/jsx-props-no-spreading` for DOM elements.
- [LINTER + NEITHER] Silent catch blocks that swallow a real failure and render an empty list. **Check:** `no-empty`; a catch that returns `[]` is judged

in review.

- [LINTER] Commented-out code kept “for reference” — git already keeps it. **Check:** `sonarjs/no-commented-code`.
- [NEITHER] A shared `utils.ts` that becomes a dumping ground. If it has thirty unrelated exports, it is not a utility.
- [NEITHER] Mapping `snake_case` to `camelCase` inside a feature. Fix the endpoint (see 1.1).
- [NEITHER] A second data-fetching approach alongside TanStack Query.
- [NEITHER] `'use client'` at the top of a `layout.tsx` or a `page.tsx`.
- [NEITHER] Business logic in a Route Handler or Server Action. It belongs in the backend service.
- [NEITHER] A server-only module imported anywhere reachable from a client component. Guard it with `server-only`.
- [NEITHER] `cache` left at its default on an authenticated fetch.

12. Money and numeric values

[NEITHER] **Never use JavaScript `number` arithmetic on a monetary or token amount.**

`0.1 + 0.2` is `0.30000000000000004`. On a financial platform that produces defects found by clients, not by us.

Kind	Type in the app	Why
Fiat currency	<code>string</code> in minor units (“1050” = ₹10.50), or a decimal library	<code>number</code> loses precision above 2^{53} and on fractions
Token / on-chain	<code>bigint</code> in base units (wei, etc.)	Exact, and matches what the chain returns

Amounts stay in base units **through the whole application**. Conversion to display units happens only at the presentation boundary.

```
// src/lib/format/amount.ts
export function formatAmount(value: string | bigint, decimals = 2, symbol = '₹'): string {
  const raw = typeof value === 'bigint' ? value : BigInt(value);
  const base = 10n ** BigInt(decimals);
  const whole = raw / base;
  const frac = (raw % base).toString().padStart(decimals, '0');
  return `${symbol}${whole.toLocaleString('en-IN')}.${frac}`;
}
```

```
<span>{formatAmount(loan.principalMinor)}</span>
<span>{formatAmount(position.amountWei, 18, 'ETH ')}</span>
```

Never:

```
const total = parseFloat(a) + parseFloat(b);    // ✗
const fee = amount * 0.025;                    // ✗
<span>{loan.principal / 100}</span>               // ✗
```

Zod schemas type money as `z.string()`, never `z.number()`.

Totals, fees and interest are calculated on the backend. The frontend displays what it is given. If the UI needs a computed total before submission, it asks the API rather than reproducing the formula — two implementations of one rule will disagree eventually.

13. Components and the UI

Most of this standard protects us from defects. This section protects us from **rework**.

Our clients change designs during a project and after launch. Whether a redesign costs three days or three weeks is decided when the components are first written, not when the new Figma file arrives. That difference is usually not billable.

13.1 The test

When the design changes, what has to change in the code?

- **Tokens, styles and markup** → the architecture is right.
- **Queries, state shapes, types or data fetching** → the separation was wrong, and we are paying for it in rework.

Ask this of your own component before you open a PR.

13.2 Tailwind is the styling system

[NEITHER] All new React projects use Tailwind. Existing projects keep the styling system they have and consume the same tokens.

Reason, in order of weight on this team:

1. **Tailwind constrains by construction.** There is no class for `13px`. A developer cannot go off-scale without arbitrary-value syntax, which is visible in review.
2. **There is no cascade to fight.** Specificity wars and `!important` escalation are the most common CSS defects on a junior team. Tailwind does not have them.
3. **Dead styles cannot accumulate.** Delete a component and its styles go with it.
4. **One system across two frameworks.** A developer moving between the Angular and React teams does not relearn styling.

Where hand-written CSS is still correct — exactly four places:

Permitted	Example
The token file	<code>tokens.css</code> — the one place raw values live
The global layer	Resets, font loading, <code>@keyframes</code> , print styles
Third-party overrides	Styling a vendor component we do not control
Existing non-Tailwind projects	Kept as-is, using the same tokens

Anything outside that list is written in Tailwind. “I needed CSS for this” is answered by the table, not by preference.

Repeated utility strings across files mean extract a component, not start a stylesheet. Use `clsx` or `tailwind-merge` for conditional classes, never string concatenation.

13.3 Design tokens are the single source of truth

[LINTER] Colours, spacing, typography, radii, shadows and breakpoints are defined once, as CSS custom properties. A component never contains a raw value.

Tokens live in `tokens.css`, not in `tailwind.config.js` — a JavaScript config is invisible to hand-written CSS, and a colour defined in two places will diverge. **This file is identical to the Angular one.** Both teams use the same tokens, which is what makes a client rebrand one change rather than two.

```
/* src/styles/tokens.css — the only file where a raw value appears */
:root {
  /* ---- Palette: raw brand values, referenced only by the semantic layer ---- */
  --brand-500: #1a6ea8;    --brand-600: #14557f;
  --neutral-0: #ffffff;    --neutral-100: #f4f6f8;
  --neutral-600: #5b6670;  --neutral-900: #13202b;
  --red-500: #c0392b;      --amber-500: #d29922;  --green-600: #2ea043;

  /* ---- Semantic: what components actually use ---- */
  --color-surface:      var(--neutral-0);
  --color-surface-muted: var(--neutral-100);
  --color-text:         var(--neutral-900);
  --color-text-muted:   var(--neutral-600);
  --color-border:       #d8dee4;
  --color-primary:      var(--brand-500);
  --color-primary-hover: var(--brand-600);
  --color-danger:        var(--red-500);
  --color-warning:       var(--amber-500);
  --color-success:       var(--green-600);

  /* ---- Spacing scale. Nothing between these steps. ---- */
  --space-1: 0.25rem; --space-2: 0.5rem;  --space-3: 0.75rem;
  --space-4: 1rem;    --space-6: 1.5rem;  --space-8: 2rem;
```

```

--space-12: 3rem; --space-16: 4rem;

/* ---- Type scale ---- */
--text-xs: 0.75rem; --text-sm: 0.875rem; --text-base: 1rem;
--text-lg: 1.125rem; --text-xl: 1.5rem; --text-2xl: 2rem;
--font-sans: 'Inter', system-ui, sans-serif;

/* ---- Radii and elevation ---- */
--radius-sm: 4px; --radius-md: 8px; --radius-lg: 16px; --radius-full: 9999px;
--shadow-1: 0 1px 2px rgb(0 0 0 / 0.06);
--shadow-2: 0 4px 12px rgb(0 0 0 / 0.10);
}

/* A theme overrides the semantic layer only – never the palette, never a component. */
[data-theme='dark'] {
  --color-surface: var(--neutral-900);
  --color-surface-muted: #1c2b38;
  --color-text: var(--neutral-0);
  --color-text-muted: #9aa7b2;
  --color-border: #2c3a46;
}

```

Two layers, and the distinction matters. The palette holds raw values. The semantic layer names a use. Components reference only the semantic layer, which is why a theme or a client rebrand is a handful of lines rather than a search-and-replace.

Tailwind v4 consumes them CSS-first:

```

/* src/styles/theme.css */
@import 'tailwindcss';
@import './tokens.css';

@theme inline {
  --color-surface: var(--color-surface);
  --color-primary: var(--color-primary);
  --color-danger: var(--color-danger);
  --spacing-1: var(--space-1);
  --spacing-4: var(--space-4);
  --radius-md: var(--radius-md);
  --font-sans: var(--font-sans);
}

```

Check: `stylelint-declaration-strict-value` on colour, spacing, font-size, radius and shadow properties, permitting only `var(...)`, `inherit`, `currentColor` and `transparent`.

13.4 Figma and code use the same component names

[NEITHER] A component's name in Figma and its name in the codebase are the same name.

They are not today. When a designer says “update the Card” nobody can map that to a file without opening several, and a design handoff becomes a translation exercise that loses detail every time.

Fixing it is one session between the lead and the designer, per project:

- Pick the name the design uses, unless it is clearly wrong. The designer renaming is usually cheaper than the developer renaming.
- `Card/Default`, `Card/Compact` in Figma → `<Card variant="compact">` in code.
- A Figma variant becomes a prop, not a second component.
- Record the mapping in the project README where a one-to-one name is impossible.

New projects start aligned. Existing projects align at the next design revision for the components that revision touches — not as a separate renaming exercise.

13.5 Three kinds of component

[NEITHER] Every component is one of these, and the kind decides what it may contain.

Kind	Lives in	May use	May do
Page	<code>features/<x>/</code>	Queries, router	Fetch data, own view state, compose children
Feature	<code>features/<x>/components/</code>	Nothing	Know the domain; typed props only
Presentational	<code>components/</code>	Nothing	Render what it is given. Knows no domain at all

A **presentational component that fetches is the single most expensive mistake in this section**. It cannot be reused, cannot be tested in isolation, and cannot survive a redesign that moves it to a different screen.

```
// ✓ Presentational. Reusable, testable, survives any redesign.
interface StatusPillProps {
  label: string;
  tone?: 'neutral' | 'success' | 'danger';
}

export function StatusPill({ label, tone = 'neutral' }: StatusPillProps) {
  return <span className={clsx('rounded-full px-3 py-1 text-sm', TONE[tone])}>{label}</span>;
}
```

```
// ✗ Not presentational. It fetches, so it can only ever be used on one screen.
export function StatusPill({ loanId }: { loanId: string }) {
  const { data } = useLoan(loanId); // ✗
}
```

Check: backstop — `no-restricted-imports` in `components/**` banning `@tanstack/react-query`, the router, and `features/*`.

13.6 Props are the contract; markup is not

[NEITHER] The internal markup and classes of a component may change at any time. **Its props may not.**

Changing or removing a prop on a shared component is a breaking change: add the new prop, migrate the consumers, then remove the old one. Do not edit it in place and fix the compile errors.

Presentational components take **view data, not domain objects**. A component typed to `Loan` breaks when the API changes; a component typed to `{ title, subtitle, tone }` does not.

13.7 A component never sets its own outer margin

[NEITHER] A component owns everything inside its box. The **parent** owns where it sits and how much space is around it.

Reason: a component carrying `mb-4` fights every reuse, and a layout redesign means editing every child rather than one parent.

```
/* ✗ */ <div className="mb-4 rounded-md p-4">...</div>
/* ✓ */ <div className="rounded-md p-4">...</div>
/* */ <div className="flex flex-col gap-4">{children}</div> // parent owns spacing
```

Use `gap` on the parent. It is the whole answer to spacing between siblings.

13.8 Styles never reach outward

[NEITHER] A component styles itself. It never styles another component's internals.

If a parent needs a child to look different, the child exposes a prop or a documented CSS custom property. A `className` prop is acceptable on a presentational component **only** when merged with `tailwind-merge`, so the component's own classes stay predictable:

```
export function Card({ className, children }: CardProps) {
  return <div className={twMerge('rounded-md bg-surface p-4 shadow-1', className)}>{children}</div>;
}
```

13.9 Responsive and accessible as it is written

[NEITHER] Mobile-first, breakpoints from tokens, accessibility built in at the time the component is written.

Retrofitting accessibility after a redesign costs several times what building it in costs, and for clients in regulated or public-sector work it is a contractual exposure. If a client contract carries an accessibility obligation, the lead confirms which standard applies before the first screen is built.

Minimum for every component:

- Semantic elements — `<button>` for actions, `<a>` for navigation. Never a `<div>` with `onClick`.
- Every input has a `<label>`; every icon-only control has an `aria-label`.
- Focus is visible and never removed with `outline-none` alone.
- Interactive targets at least 44×44px on touch.
- Text contrast meets 4.5:1 — checked against the tokens once, not per screen.

Check: `eslint-plugin-jsx-a11y` catches the mechanical half. The rest is checklist item 14.

13.10 Starting the shared component library

We do not have one today. The way to get one is not to sit down and design it.

[NEITHER] A component graduates to `components/` on its third use, not its first. Two uses is a coincidence; three is a pattern, and by then you know which parts actually vary.

When it graduates it gains a named owner, a documented prop contract, a test if it holds logic, and the breaking-change rule in 13.6.

Keep it as a folder inside one repo. **Do not extract it into an npm package until two repositories genuinely need the same component** — a published package adds versioning, release and upgrade work, and we have one frontend lead.

13.11 Visual regression — next, not now

[NEITHER] A token change can alter thirty screens silently. A screenshot baseline catches that; nothing else does.

This needs tooling we do not have, so it is a planned improvement rather than a day-one rule. Until it exists, a PR that changes `tokens.css` names in its description which screens were checked by hand.

14. Server state

14.0 Which mechanism, and when

[NEITHER] Two ways to get server data, and the choice is not a preference.

Use	When
Server Component fetch (8.3)	Initial page data that does not depend on client interaction — a list on first load, a detail page, anything that should be in the HTML
TanStack Query	Anything driven by interaction — mutations, filtering, pagination, polling, infinite scroll, optimistic updates, wallet-dependent data

They coexist on one page. The server fetches the first view; the client takes over from there. Where both cover the same data, the server result seeds the query cache with `HydrationBoundary` so the client does not refetch what the server already sent.

All client-side server data goes through TanStack Query. Every new project, no exceptions.

Reason: server state is a different problem from client state. It is owned by someone else, it goes stale, it arrives twice, it fails halfway. A team that handles it with `useState` and `useEffect` ends up hand-rolling caching, deduplication, retry and invalidation — badly, and differently in every project. Most React defects on a junior team live in that gap.

`useState` and `useReducer` remain correct for **client** state: form drafts, open/closed, selected tab, wizard step.

14.1 Query keys are defined once, per feature

[NEITHER] Never write a query key inline. A mutation that invalidates `['loans']` while a query registered `['loan', 'list']` silently does nothing, and the bug looks like a stale UI rather than a typo.

```
// src/features/loans/api/keys.ts
export const loanKeys = {
  all: ['loans'] as const,
  lists: () => [...loanKeys.all, 'list'] as const,
  list: (filters: LoanFilters) => [...loanKeys.lists(), filters] as const,
  details: () => [...loanKeys.all, 'detail'] as const,
  detail: (id: string) => [...loanKeys.details(), id] as const,
};
```

The hierarchy is the point: invalidating `loanKeys.all` clears every loan query, `loanKeys.lists()` clears only the lists.

14.2 Queries live in the feature's `api/` folder

[NEITHER] A component never calls `useQuery` directly. It calls a named hook.

```
// src/features/loans/api/use-loans.ts
import { useQuery } from '@tanstack/react-query';

export function useLoans(filters: LoanFilters) {
  return useQuery({
    queryKey: loanKeys.list(filters),
    queryFn: () => apiFetch('/loans', { schema: loanListSchema }),
    staleTime: 30_000,
  });
}
```

Schema validation happens inside `apiFetch`, so `data` is typed and verified by the time any component sees it.

14.3 Mutations invalidate what they changed

[NEITHER] Every mutation declares what it invalidated. A mutation that changes server data and invalidates nothing leaves a UI showing values that are no longer true.

```
export function useSettleLoan() {
  const queryClient = useQueryClient();

  return useMutation({
    mutationFn: (id: string) =>
      apiFetch(`/loans/${id}/settle`, { method: 'POST', schema: loanSchema }),

    onSuccess: (loan) => {
      queryClient.setQueryData(loanKeys.detail(loan.id), loan); // we have the new value
      void queryClient.invalidateQueries({ queryKey: loanKeys.lists() });
    },
  });
}
```

Optimistic updates are used only where the operation is near-certain to succeed and the rollback is simple. On financial operations, wait for the server. A balance that appears then reverts is worse than one that takes 400ms.

14.4 Client defaults

In Next, the `QueryClient` is created **inside** the client provider with `useState`, never at module scope. A module-scoped client is shared between requests on the server and leaks one user's data into another's response.

```
// src/lib/query/client.ts – called from Providers via useState(() => makeQueryClient())
export function makeQueryClient() {
  return new QueryClient({
    defaultOptions: {
      queries: {
        staleTime: 30_000, // don't refetch on every mount
        gcTime: 5 * 60_000,
        retry: (failureCount, error) => {
          // Never retry a client error – the request was wrong, not unlucky.
          if (error instanceof ApiError && error.status >= 400 && error.status < 500) return false;
          return failureCount < 2;
        }
      }
    }
  });
}
```

```

    },
    refetchOnWindowFocus: false,    // turn on per-query where live data matters
  },
  mutations: { retry: false },      // a retried write is not a harmless retry
},
});
}

```

Mutations never retry by default. A retried POST can create two records. Where a retry is genuinely safe, the endpoint takes an idempotency key and the hook opts in explicitly.

14.5 What this replaces

- [NEITHER] No `useEffect` with a `fetch` inside it. That is a query.
- [NEITHER] No manual `isLoading` / `error` / `data` state triple. The query returns them.
- [NEITHER] No global store holding server data. Redux, Zustand and Context hold **client** state; TanStack Query owns server state.
- [NEITHER] No second data-fetching library alongside it.
- [NEITHER] No `QueryClient` at module scope. It is created per request inside the client provider.

Appendix — ESLint starting point

Baseline existing violations rather than fixing them all at once: suppress what exists today and fail the build only on new violations.

```

// eslint.config.js (flat config)
export default [
  {
    files: ['src/**/*.ts', 'src/**/*.tsx'],
    rules: {
      'react-hooks/rules-of-hooks': 'error',
      'react-hooks/exhaustive-deps': 'error',    // error, not warn
      '@typescript-eslint/no-explicit-any': 'error',
      '@typescript-eslint/no-non-null-assertion': 'error',
      '@typescript-eslint/no-unused-vars': 'error',
      '@typescript-eslint/no-magic-numbers': ['warn', { ignore: [0, 1, -1] }],
      'no-console': 'error',
      'max-lines-per-function': ['error', 75],
      'max-depth': ['error', 3],
      'max-lines': ['error', 250],
      'no-empty': 'error',
      'sonarjs/no-commented-code': 'error',
      'react/jsx-key': 'error',
      'react/no-array-index-key': 'error',
      'react/no-danger': 'error',
      'react/jsx-handler-names': 'error',
      'react/jsx-props-no-spreading': ['error', { html: 'enforce', custom: 'ignore' }],
      'no-restricted-globals': ['error',
        { name: 'fetch', message: 'Use apiFetch from lib/api. Standard 3.2.' },
      ],
      'no-restricted-properties': ['error',
        { object: 'localStorage', property: 'setItem' },
        { object: 'sessionStorage', property: 'setItem' },
      ],
      'no-restricted-syntax': ['error', {
        selector: 'MemberExpression[object.meta.name=\'import\'][object.property.name=\'meta\']',
        message: 'Read config through lib/config. Standard 8.2.',
      }],
    },
  },
  {
    files: ['src/lib/api/**/*.ts', 'src/lib/config/**/*.ts'],
    rules: { 'no-restricted-globals': 'off', 'no-restricted-syntax': 'off' },
  },
  {
    files: ['src/lib/logging/**/*.ts'],
    rules: { 'no-console': 'off' },
  },
  {
    // Presentational components stay presentational (13.5).
    files: ['src/components/**/*.tsx'],
    rules: {
      'no-restricted-imports': ['error', {
        paths: [

```

```

    { name: '@tanstack/react-query', message: 'Presentational components do not fetch. Standard 13.5.' },
    { name: 'react-router-dom', message: 'Presentational components do not navigate. Standard 13.5.' },
  ],
  patterns: [{ group: ['**/features/**'], message: 'Shared components know no domain. Standard 13.5.' }],
}],
},
},
];

```

Add for Next:

```

{
  // app/ is routing only (section 2).
  files: ['src/app/**/*.tsx'],
  rules: {
    'no-restricted-imports': ['error', {
      patterns: [{ group: ['**/lib/api/**'],
        message: 'Pages compose features. Data access lives in features/*/api. Standard 2.' }],
    }],
  },
},
{
  // Server-only modules must announce themselves (8.1).
  files: ['src/lib/server/**/*.ts'],
  rules: {
    'no-restricted-syntax': ['error', {
      selector: "Program:not(:has(ImportDeclaration[source.value='server-only']))",
      message: "Every file in lib/server must import 'server-only'. Standard 8.1.",
    }],
  },
},
},

```

Also required: `eslint-config-next` (which carries `@next/next/*` rules), `eslint-plugin-check-file` for file naming, `eslint-plugin-boundaries` for cross-feature imports, `eslint-plugin-testing-library` for tests, and `eslint-plugin-jsx-a11y` for section 13.9.

`@next/next/no-html-link-for-pages` and `no-img-element` are errors, not warnings.

Appendix — Stylelint starting point

Enforces the token rules in section 13.3. Identical to the Angular configuration.

```

// .stylelintrc.js
module.exports = {
  extends: ['stylelint-config-standard'],
  plugins: ['stylelint-declaration-strict-value'],
  rules: {
    'scale-unlimited/declaration-strict-value': [
      ['/color$/'], 'fill', 'stroke', 'padding', 'margin', 'gap',
      'font-size', 'border-radius', 'box-shadow',
      {
        ignoreValues: ['inherit', 'currentColor', 'transparent', 'none', 'auto', '0'],
        disableFix: true,
        message: 'Use a token from tokens.css. See standard 13.3.',
      },
    ],
    'declaration-no-important': true,
  },
  ignoreFiles: ['src/styles/tokens.css'],
};

```

`tokens.css` is the one file exempt from the rule, because it is where raw values are allowed to exist.

Appendix C — React without Next

Some projects are plain React on Vite. Every section applies except where Next supplies the mechanism. The substitutions:

Rule	Next	Plain React
Routing & structure (2)	<code>app/</code> directory	React Router with <code>lazy()</code> per route; <code>features/</code> unchanged
<code>'use client'</code> (2.1)	The boundary rule	Not applicable — everything is client code
Errors (3.4)	<code>error.tsx</code> , <code>global-error.tsx</code>	<code><ErrorBoundary></code> from <code>react-error-boundary</code> , one per route
Loading (3.4)	<code>loading.tsx</code>	The query's <code>isPending</code> branch
Public env (8.1)	<code>NEXT_PUBLIC_*</code>	<code>VITE_*</code> — same rule, same exposure
Config (8.2)	Server reads <code>process.env</code> , passes <code>publicConfig()</code> down	Fetch <code>/config.json</code> before <code>createRoot(...).render(...)</code>
Server fetch (8.3)	<code>cookies()</code> + <code>serverFetch</code>	Not available; all data is client-fetched
Proxy endpoints (8.6)	Route Handler	An endpoint on the Node or Python service
Server state (14)	Server Components + TanStack Query	TanStack Query for everything

Everything else — naming, wire format, logging, redaction, Sentry, testing, PR rules, money, components and tokens — is identical and is not restated here.